

**Министерство науки и высшего образования
Российской Федерации**

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)**

Институт информатики и кибернетики

Кафедра информационных систем и технологий

**Пояснительная записка к курсовому проекту по курсу
«Микропроцессорные средства и системы»
Таймер на основе микроконтроллера Arduino UNO R3**

Выполнил студент Тиликанов С. В

4 Курс, группа 6496-090301z

Номер зачетной книжки 2021-04744

Преподаватель Семенов В. В.

Работа сдана на проверку _____
дата *подпись*

Регистрационный номер _____

Работа проверена _____
дата *подпись*

Оценка _____ Подпись _____

Самара 2025

Оглавление

1.	Анализ технического задания	3
2.	Выбор микроконтроллера и комплектующих	4
2.1.	Основные сведения об Arduino Uno R3	4
2.2.	Основные сведения об ATmega328P	5
2.3.	Семисегментный дисплей и периферия	6
3.	Разработка программного обеспечения	8
	Заключение	18
	Приложение 1. Исходный код программы	19
	Приложение 2. Демонстрация работы устройства в симуляторе Proteus	23
	Приложение 3. Демонстрация работы собранного прототипа устройства	26
	Список использованных источников	29

1. Анализ технического задания

Согласно поставленной задаче, мы должны собрать таймер на базе микроконтроллера Arduino и семисегментного индикатора. Реализуем следующую логику работы алгоритма:

- 1) Пуск таймера будет осуществляться с помощью кнопки. При повторном нажатии отсчет времени будет приостанавливаться.
- 2) Время и его отсчет будет отображаться на семисегментном индикаторе с точностью до десятой доли секунды.
- 3) По истечении времени будет загораться светодиод.
- 4) Добавим к этому возможность регулировать время отсчета в сторону увеличения и уменьшения с помощью еще двух кнопок. При зажиме этих кнопок изменение времени отсчета будет постепенно ускоряться (секунда – 5 секунд – 10 секунд – минута – 5 минут – 10 минут).
- 5) Также, время таймера будет сохраняться в энергонезависимую память, чтобы после сброса или выключения МК заново его не восстанавливать.

Программный код будет писаться на языке C и компилироваться в IDE Arduino

Разработка и тестирование будет вести с помощью пакета Proteus для проектирования и симуляции работы электрических схем и микроконтроллеров.

После завершения виртуального тестирования и отладки, устройство будет собрано на макетной плате BreadBoard.

2. Выбор микроконтроллера и комплектующих

Изначально выбор пал на МК ATmega8 в силу его простоты и дешевизны, а также несложности самой программы. Но я столкнулся с непредвиденными сложностями при его программировании: перепрошивка МК завершалась успешно в 1 случае из 5, а в остальных случаях завершалась со всевозможными ошибками, например, ошибкой тактирования. Честно говоря, я не имею представления с чем может быть связана эта неудача. Впрочем, микроконтроллеры продаются с высоким процентом брака при производстве, а заблокированные и поврежденные МК зачастую перепродаются заново ничего не подозревающим покупателям. Не лучше обстоит дело и с программаторами: они часто продаются со старыми прошивками, в связи с чем требуется уже программатор для программатора для перезаписи.

После безрезультатного приобретения двух программаторов, двух микроконтроллеров, одной отладочной платы, было решено обратить внимание на более отзывчивое к пользователю решение: МК серии Arduino.

2.1. Основные сведения об Arduino Uno R3

Arduino Uno Rev3 — это плата, основанная на микроконтроллере ATmega328P. Платформа имеет 14 цифровых пинов входа/выхода, 6 из которых могут использоваться как выходы ШИМ, 6 аналоговых входов, кварцевый генератор 16 МГц, разъем USB, силовой разъем, разъем ICSP и кнопку перезагрузки. Для работы необходимо подключить платформу к компьютеру посредством кабеля USB, либо подать питание при помощи адаптера AC/DC или батареи. Китайская версия Arduino Uno Rev3 в качестве преобразователя интерфейсов USB-UART использует микроконтроллер CH340G [1].

Чтобы включить плату, нужно на неё подать питание либо от USB порта, можно прямо от ПК, либо от внешнего источника питания – от 7 до 15 Вольт. На плате установлен линейный стабилизатор, типа L7805, или же LDO. Он

нужен для того, чтобы на микроконтроллер подавалось стабилизированное напряжение 5 В.

При этом приоритетно выбирается внешний источник питания, а не ЮСБ-порт. Внешнее питание подключается к выводу с пометкой «Vin» в разделе Power на плате [2].

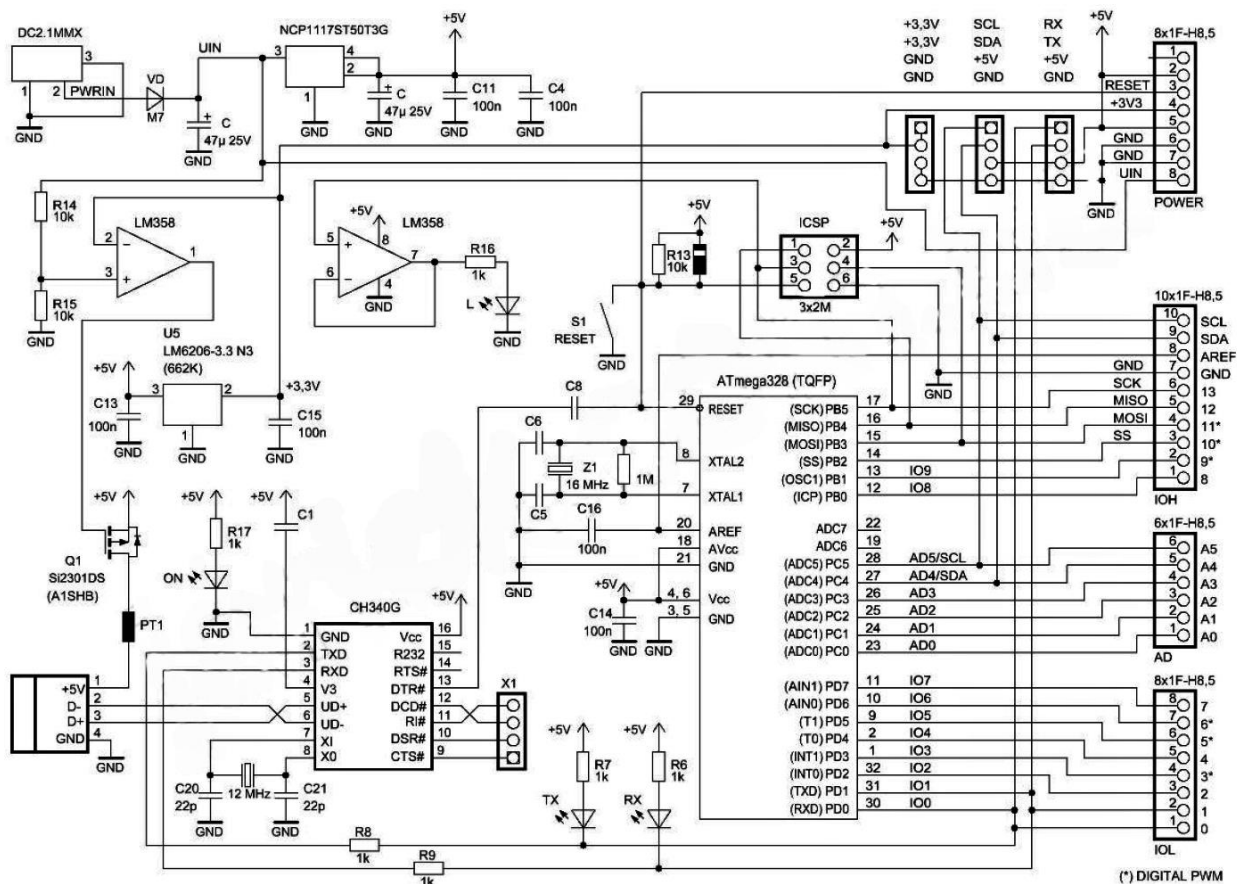


Рис 1. Принципиальная схема Arduino UNO R3

2.2. Основные сведения об ATmega328P

ATMEGA328P — это 8-битный CMOS -микроконтроллер с низким энергопотреблением, основанный на RISC-архитектуре с усовершенствованием AVR. ATmega328P предоставляет следующие возможности:

- 32 Кб внутрисистемной программируемой флэш-памяти с возможностью чтения во время записи;
- 1 Кб энергонезависимой памяти EEPROM; с продолжительностью 100 000 циклов стирания и записи;

- 2 КБ SRAM, служащей хранилищем для основных данных или временного кэширования задач во время выполнения.,
- 32 рабочих регистра общего назначения, которые функционируют в полностью статическом рабочем режиме, обеспечивая улучшенную скорость микроконтроллера и снижение энергопотребления;
- три гибких таймера/счетчика с режимами сравнения, внутренними и внешними прерываниями: два 8-битных и один 16-битный таймер/счетчик;
- последовательный программируемый USART и т. д.

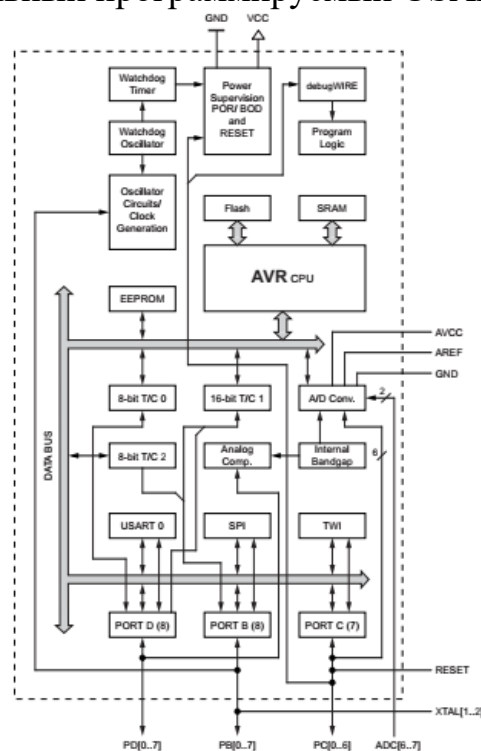


Рис 2. Блок-схема ATMEGA328-P

2.3. Семисегментный дисплей и периферия

Технические характеристики семисегментного катодного индикатора 5461AR:

- Цвет: Красный
- Количество цифр: 4
- Высота цифры: 14.2мм (0.56")
- Прямое падение напряжения на сегмент: 2...2.3В
- Прямой ток на сегмент: 20мА

- Обратный ток на сегмент: 20мкА
- Сила света на сегмент: 14мкД
- Схема соединения сегментов: общий катод
- Диапазон температур: -30...+85С
- Размеры: 50.3x19x8мм [3]

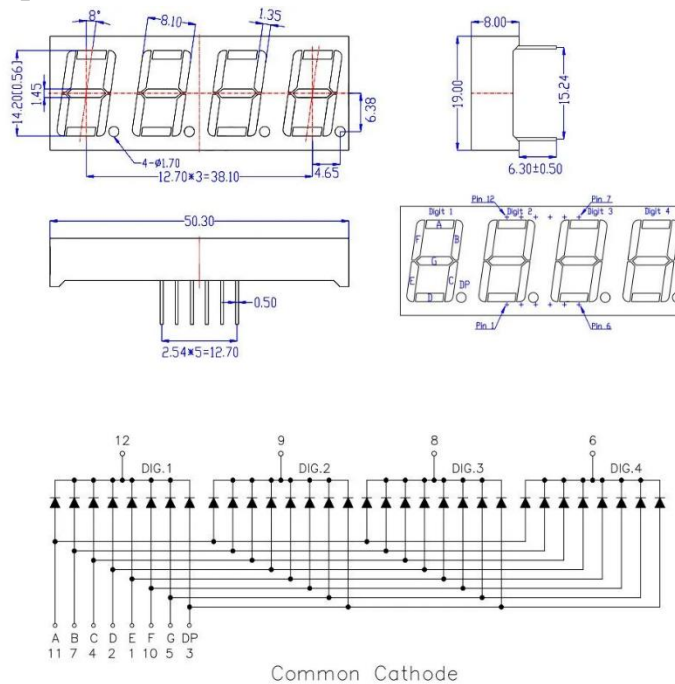


Рис 3. Схема семисегментного индикатора 5461AR

Помимо индикатора нам понадобятся 3 кнопки, 1 светодиод, набор резисторов номиналом 150-200 Ом, провода, макетная плата BreadBoard.

3. Разработка программного обеспечения

Соберем принципиальную схему таймера в Proteus.

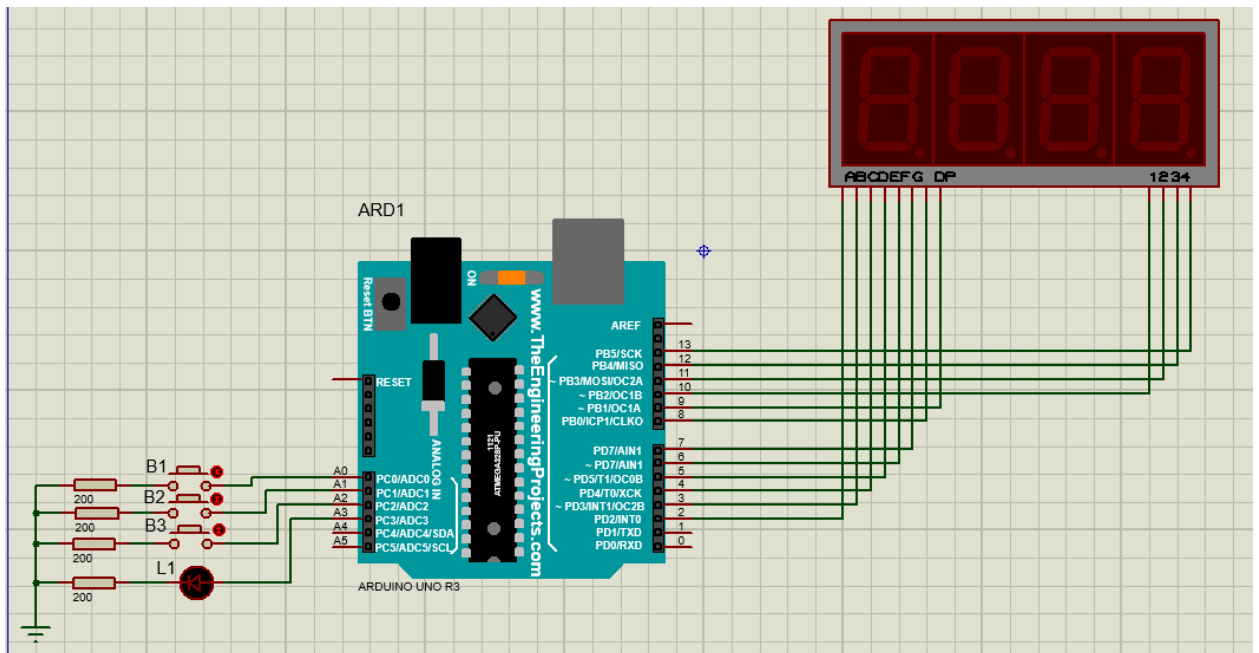


Рис 4. Принципиальная схема таймера в Proteus

```
# define F_CPU 8000000UL
```

Настраиваем тактовую частоту МК на 8МГц.

```
#include <avr/io.h>
#include <avr/interrupt.h>
```

Подключаем необходимые нам библиотеки

```
const int LEDD[] = {0b11111100, 0b00011000, 0b01101100, 0b00111100,
0b10011000, 0b10110100, 0b11110100, 0b00011100, 0b11111100, 0b10111100};
const int LEDB[] = {0b00000000, 0b00000000, 0b00000001, 0b00000001,
0b00000001, 0b00000001, 0b00000001, 0b00000000, 0b00000001, 0b00000001};
```

PD0 и PD1 задействованы для работы с интерфейсами поэтому задействовать их нельзя. Поэтому разведем сегменты дисплея по разным портам.

6 сегментов подсоединены к порту D (PD2 – PD7), и массив LEDD[] отвечает за корректное отображение сегментов A-F. Числа в массивах отображают, какие выходы должны подавать высокое напряжение, а какие – нет. Сегменты G, DP подсоединены к порту B, и массив LEDB[] отвечает за корректное их отображение.

Таким образом, например, для отображения числа 6 на дисплее нам необходимо подать на порты B и D соответствующие значения массивов.

```
PORTD |= LEDD[6];
PORTB |= LEDB[6];
```



```
struct time {
    char minutes;
    char seconds;
    char milliseconds;
} ;
```

Объявляем структуру типа «время», которая позволит хранить 3 поля (минуты, секунды, миллисекунды в одном объекте).

```
int millis = 0;
```

Объявляем переменную millis, в которой будем хранить время отсчета в десятых долях секунды.

Метод init() производит предварительную настройку регистров, таймеров и портов для корректной работы.

```
void init() {
    DDRD = ~0x03;
    DDRB = 0b00111111;
```

Настройка на вывод пинов PD2-PD7, PB0-PB5. Они будут подключены к индикатору.

```
    DDRC = 0b00001000;
    PORTC = 0x07;
```

Настройка пинов PC0-PC2 на прием (будут подключены к кнопкам), пина PC3 на вывод (будет подключен к светодиоду).

```
    sei();
```

Разрешение прерываний .

```
    TCCR0B=0b00000001;
    TCNT0 = 0;
    TIMSK0|=(1<<TOIE0);
```

Настройка таймер-счетчика 0. Он будет отвечать за корректное отображение цифр на индикаторе.

В регистре TCCR0B выставляем коэффициент делителя равный 1

Выставляем начальное значение счетного регистра TCNT0 равным 0.

В регистре TIMSK0 разрешаем прерывание по событию «переполнение» (Бит TOIE0). Таким образом, каждый раз при переполнении регистра TCNT0 (максимальное достижимое значение - 255), будет срабатывать прерывание, которое мы должны будем обработать.

```
    //TCCR1B = 0b00001011;
    TIMSK1|=(1<<OCIE1A);
    TCNT1 = 0;
    OCR1A = 6250;
```

Настройка таймер-счетчика 1. Он будет отвечать за корректный обратный отсчет нашего устройства.

Выставляем начальное значение счетного регистра TCNT1 равным 0.

В регистре TIMSK1 разрешаем прерывание по событию «совпадение» (Бит OCIE1A). Значение регистра сравнения OCR1A сделаем равным 6250.

Каждый раз, когда регистр TCNT1 будет достигать этого значения, будет срабатывать переполнение. Значение OCR1A было подобрано таким образом, чтобы срабатывание прерывания наступало раз в десятую долю секунды.

Настройки завершены, но до тех пор, пока мы не активируем регистр TCCR1B, счетчик будет остановлен.

```
//TCCR2B = 0b00000011;  
TCNT2 = 0;  
TIMSK2 |= (1 << TOIE2);
```

Настройки таймер-счетчика 2 аналогичны настройкам таймер-счетчика 1, однако и он тоже пока остановлен.

```
return_number();  
}
```

Вызываем метод return_number() который загружает из EEPROM сохраненное значение таймера. Ознакомимся с ним, когда разберем метод записи данных в EEPROM.

Рассмотрим метод return_capacity(unsigned int num) в котором раскладываем значение переменной num на 4 десятизначные цифры, которые затем будут выводиться на экран:

```
void return_capacity(unsigned int num){  
    struct time t = convert_millis_to_time(num);
```

Переменную num разложим по значениям полей структуры time (минуты, секунды, десятые доли секунды) с помощью метода convert_millis_to_time(unsigned int num).

```
less_10_min = (t.minutes < 10);
```

Введем флаг less_10_min, который показывает, является ли количество минут таймера двуразрядным либо одноразрядным числом.

```
if(less_10_min) {  
    digits[0] = t.minutes;  
    digits[1] = t.seconds / 10;  
    digits[2] = t.seconds % 10;  
    digits[3] = t.milliseconds;  
}
```

Если количество минут одноразрядное, то в массив `digits[]` последовательно записываются: количество минут, десятичный разряд секунд, единичный разряд секунд, десятые доли секунды.

```
else{
    digits[0] = t.minutes / 10;
    digits[1] = t.minutes % 10;
    digits[2] = t.seconds / 10;
    digits[3] = t.seconds % 10;
}
```

В противном случае записываются: десятичный разряд минут, единичный разряд минут, десятичный разряд секунд, единичный разряд секунд.

Метод `convert_millis_to_time(unsigned int num)`:

```
struct time convert_millis_to_time(unsigned int num){
    struct time result;
```

Создаем переменную типа `time`, в которую запишем результат.

```
    result.milliseconds = num % 10;
```

В поле `milliseconds` запишем остаток от деления числа `num` на 10.

```
    num /= 10;
```

Делим число `num` на 10.

```
    result.seconds = num % 60;
```

В поле `seconds` запишем остаток от деления числа `num` на 60.

```
    result.minutes = num / 60;
```

В поле `minutes` запишем результат деления числа `num` на 60.

```
    return result;
}
```

Возвращаем переменную `result`.

Рассмотрим обработчик прерывания таймера 0:

```
ISR(TIMERO_OVF_vect){
    i = ( i + 1 ) % 4;
```

Переменная-итератор `i` принимает значения от 0 до 3.

```
    PORTD &= 0x03;
```

Предварительно выставляем в 0 выходы PD2-PD7 и гасим все сегменты дисплея.

```
    PORTB = ~(1 << (i+2)) & 0b00111100;
```

С той же целью выставляем в 0 выходы PB0, PB1. Порты PB2-PB5, напротив, выставляем в 1. Так как дисплей катодный, то для активации разряда требует низкого сигнала. Порт `PB[i+2]` выставляем в 0.

```
    PORTD |= LEDD[digits[i]];
    PORTB |= LEDB[digits[i]];
```

Выводим на `i`-е место на дисплее `i`-ю цифру из массива `digits[]`.

```

        if((less_10_min && ((i == 2) || (i == 0)))
        || (!less_10_min && i == 1))
        PORTB |= (1<<i);
    }
}

```

Если минуты таймера - одноразрядное число, то ставим точку на четвертом и на втором разряде дисплея (Отделяем минуты от секунд, а секунды от десятых долей). Если минуты таймера – двухразрядное число, то ставим точку на третьем разряде и отделяем минуты от секунд.

Рассмотрим метод обработки нажатия кнопок B1 и B2 -

button_handler(char port, char new_multipl):

```
void button_handler(char port, char new_multipl){
```

Переменная port означает порт, который мы «прослушиваем». Если port = 0, то мы обрабатываем нажатие кнопки B1, соединенной с портом PC0. Если port = 1, то кнопка – B2, а порт PC1. Переменная new_multipl означает множитель, с которым мы будем прибавлять coeff к значению таймера. Если new_multipl = 1, то значение мы будем прибавлять. А если new_multipl = -1, то, напротив, убавлять.

```
if(~PINC&(1<<port)){
```

Проверяем, нажата ли кнопка. Если нажата, то:

```
multiplier = new_multipl;
```

Переменная multiplier участвует в обработке Таймер-Счетчика 2. Присвоим ей значение new_multipl.

```
return_capacity(millis);
```

Обновим массив digits[]

```
TCCR2B = 0b00000011;
```

Запускаем таймер-счетчик 2 с предделителем 64.

```
while(~PINC&(1<<port))
;
```

Ожидаем отжатия кнопки.

```
counter = 0;
```

Обнуляем переменную counter участвующую в обработке таймер-счетчика 2.

```
TCCR2B = 0x00;
```

Останавливаем таймер-счетчик 2.

```
write_digit();
```

```

    }
}

```

Массив digits[] записываем в EEPROM.

Рассмотрим обработчик прерывания таймера 2:

```
ISR(TIMER2_OVF_vect){  
    counter++;
```

Так как счетчик таймера 2 не очень большой, ведем переменную-счетчик counter, с помощью которой увеличивать период прерывания и контролировать ускорение добавления (убавления) времени.

```
    if(multiplier == -1 && millis == 0)  
        return;
```

Если в режиме убавления времени millis сократились до 0, прекратить обработку прерывания.

```
    if (millis >= 54000) return;
```

Если количество миллисекунд не меньше 54000 (90 минут), прекратить прерывание.

```
    if (coeff > millis && multiplier == -1){  
        millis = 0;  
        return_capacity(millis);  
        return;  
    }
```

Если в режиме убывания коэффициент убавления времени больше самой переменной millis, то обнуляем переменную millis, чтобы избежать переполнения, обновляем массив digits[] и выходим из обработки прерывания.

```
    if(counter != 1 && counter % 200 != 0) return;
```

Добавление времени будет происходить только каждое двухсотое переполнение, либо когда кнопка только нажата и только что наступило первое прерывание.

```
    if(counter == 1)  
        coeff = 10;
```

Сначала время будет изменяться на 1 секунду.

```
    else if (counter == 1000)  
        coeff = 50;
```

Спустя 5 переключений (~ 1.5 – 2 секунды) коэффициент увеличится до 5 секунд.

```
    else if(counter == 1600)  
        coeff = 100;
```

Спустя 4 переключения (~1 – 1.5 секунд) коэффициент увеличится до 5 секунд.

```

else if(counter == 2400)
    coeff = 600;

```

Спустя 8 переключений (~2 – 3 секунды) коэффициент увеличится до 1 минуты.

```

else if(counter == 3200)
    coeff = 3000;

```

Спустя 4 переключения (~1 – 1.5 секунд) коэффициент увеличится до 5 минут.

```

else if(counter == 3800)
    coeff = 6000;

```

Спустя 6 переключений (~ 1.5 – 2 секунды) коэффициент увеличится до 10 минут.

```

millis = millis + coeff*multiplier;

```

Прибавляем к millis коэффициент coeff с множителем multiplier

```

return_capacity(millis);
}

```

Обновим массив digits[].

Рассмотрим EEPROM_write() – метод записи данных в EEPROM:

```

void EEPROM_write(unsigned int uiAddress, unsigned char ucData){

```

Переменная uiAddress отображает адрес в который будут записаны данные ucData.

```

while(EECR & (1<<EEWE))
;

```

Ожидаем завершения предыдущей записи. Бит EEWE регистра EECR должен быть сброшен до 0.

```

EEAR = uiAddress;
EEDR = ucData;

```

Записываем значения в регистр адреса и регистр данных.

```

EECR |= 1<< EEMWE;

```

Разрешаем запись данных в EEPROM.

```

EECR |= 1<<EEWE;

```

```

}

```

Начало записи данных.

Рассмотрим EEPROM_read() – метод чтения данных из EEPROM.

```

unsigned char EEPROM_read(unsigned int uiAddress){

```

Переменная uiAddress обозначает адрес, из которого будем читать данные.

```

while(EECR & (1<<EEPE))
;

```

Ожидаем завершения предыдущей записи

```

EEAR = uiAddress;

```

Устанавливаем адрес EEPROM

```
EECR |= (1<<EERE);
```

Начало чтения данных из EEPROM

```
return EEDR;
```

```
}
```

Возвращение данных из регистра.

Рассмотрим write_digit() - метод записи массива digits[] в EEPROM:

```
void write_digit(void)
{
    unsigned char r = 0;
```

Для записи в EEPROM будем использовать переменную r.

```
if(less_10_min) r = 0x0A;
else r = 0x0B;
```

Первым делом запишем в EEPROM сведения о том, больше ли таймер 10 минут или нет. Для этого сохраняем флаг less_10_min в r.

```
cli();
```

Запрещаем прерывания на время записи.

```
EEPROM_write(0, r);
```

Записываем данные из r в нулевую ячейку.

```
sei();
```

Разрешаем прерывания.

```
for(int unsigned j = 1; j<5; j++){
    r = digits[j-1];
    cli();
    EEPROM_write(j, r);
    sei();
}
```

В цикле for записываем данные из массива digits[] в ячейки EEPROM с первой по четвертую.

Рассмотрим return_number() – метод чтения массива digits[] из EEPROM.

```
void return_number(){
    unsigned char flag = 0;
    unsigned char r = 0;
    cli();
```

Запрещаем прерывания на время чтения.

```
flag = EEPROM_read(0);
```

В переменную flag будем записывать информацию из первой ячейки EEPROM, хранящую информацию о переменной less_10_min.

```
sei();
```

Снова разрешаем прерывания.

```
for(unsigned int j = 1; j<5; j++){
```

Запускаем цикл из 4 итераций.

```
cli();
r = EEPROM_read(j);
```

В переменную `r` будем записывать информацию из ячеек 0x01-0x04. Там хранятся числа из массива `digits[]`

```
sei();
digits[j-1] = r;
}
```

Пересохраняем эти числа в `digits[]`

```
millis = 0;
```

Теперь по числам из `digits[]` нам надо восстановить значение таймера `millis`.

```
if(flag == 0x0A){
    millis += digits[3];
    millis += digits[2]*10;
    millis += digits[1]*100;
    millis += digits[0]*600;
}
```

Если `less_10_min` был равен 1, то тогда пересчитываем `millis` так, чтобы младший разряд `digits[]` был равен десятой части секунды, а старший разряд был равен количеству минут.

```
else if (flag == 0x0B){
    millis += digits[3] * 10;
    millis += digits[2] * 100;
    millis += digits[1] * 600;
    millis += digits[0] * 6000;
}
```

В противном случае тогда пересчитываем `millis` так, чтобы младший разряд `digits[]` был равен единичному разряду секунды, а старший разряд был равен десятичному разряду минут.

```
return_capacity(millis);
}
```

Если флаг `flag` не принимает значения 0x0A и 0x0B то считаем, что EEPROM пуст, тогда записываем в `digits[]` нулевое значение `millis`.

Рассмотрим точку входа в программу – метод `main`:

```
int main(void)
{
    init();
    while (1)
    {
```

С помощью метода `init()` производим начальную настройку регистров, а затем входим в бесконечный цикл.

```
button_handler(0, 1);
button_handler(1, -1);
```

«Прослушиваем» порты PC0, PC1 на нажатие кнопки.


```
if(~PINC&(1<<2)){
```

Теперь проверим, нажата ли кнопка В3, соединенная с портом РС2.

```
if(TCCR1B == 0x00){
```

Если нажата, проверяем, запущен ли таймер 1.

```
    if(millis>0){
        PORTC&=~(1<<3);
        TCCR1B = 0b00001011;
    }
}
```

Если таймер не запущен, погасим светодиод L1, связанный с РС3, на случай если горит.

```
else
    TCCR1B = 0x00;
```

Если же таймер, напротив, запущен - останавливаем.

```
    while(~PINC&(1<<2))
        ;
    }
}
```

Ждем отжатия кнопки В3.

Рассмотрим обработчик прерывания таймера 1:

```
ISR(TIMER1_COMPA_vect){
    if(!millis){
        PORTC|=(1<<3);
        TCCR1B = 0x00;
        return;
    }
}
```

Если время на millis показывает 0, то тогда зажигаем светодиод L1, связанный с РС3, останавливаем таймер 1, и выходим из прерывания.

```
    return_capacity(--millis);
}
```

В противном случае уменьшаем millis на единицу и обновляем digits[].

Заключение

В ходе выполнения данной работы нами было разработано устройство основанное на микроконтроллере Arduino Uno R3, реализующее функционал таймера

Приложение 1. Исходный код программы.

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <avr/interrupt.h>

const int LEDD[] = {0b11111100, 0b00011000, 0b01101100, 0b00111100,
0b10011000, 0b10110100, 0b11110100, 0b00011100, 0b11111100, 0b10111100};
const int LEDB[] = {0b00000000, 0b00000000, 0b00000001, 0b00000001,
0b00000001, 0b00000001, 0b00000001, 0b00000000, 0b00000001, 0b00000001};
int digits[4];
unsigned int counter = 0;
char multiplier = 1;
int coeff = 10;
int millis = 0;
char less_10_min = 0;

struct time {
    char minutes;
    char seconds;
    char milliseconds;
};

struct time convert_millis_to_time(unsigned int);

struct time convert_millis_to_time(unsigned int num){
    struct time result;
    result.milliseconds = num % 10;
    num /= 10;
    result.seconds = num % 60;
    result.minutes = num / 60;
    return result;
}

void EEPROM_write(unsigned int uiAddress, unsigned char ucData){
    while(EECR & (1<<EEPE))
        ;
    EEARH = ((uiAddress & 0xF0) << 2);
    EEARL = uiAddress & 0x0F;
    EEDR = ucData;

    EECR |= 1<< EEMPE;
    EECR |= 1<<EEPE;
}

unsigned char EEPROM_read(unsigned int uiAddress){
    while(EECR & (1<<EEPE))
        ;
    EEARH = ((uiAddress & 0xF0) << 2);
    EEARL = uiAddress & 0x0F;

    EECR |= (1<<EERE);
    return EEDR;
}

ISR(TIMER0_OVF_vect){
    for (int i = 0; i<4; i++)
    {
        PORTD &= 0x03;
        PORTB=~(1<<(i+2))&0b00111100;
        PORTD |= LEDD[digits[i]];
        PORTB |= LEDB[digits[i]];
    }
}
```

```

        if((less_10_min && ((i == 2) || (i == 0))) || (!less_10_min && i == 1))
            PORTB |= (1 << i);
    }
}

void return_capacity(unsigned int num){
    struct time t = convert_millis_to_time(num);
    less_10_min = (t.minutes < 10);
    if(less_10_min) {
        digits[0] = t.minutes;
        digits[1] = t.seconds / 10;
        digits[2] = t.seconds % 10;
        digits[3] = t.milliseconds;
    }
    else{
        digits[0] = t.minutes / 10;
        digits[1] = t.minutes % 10;
        digits[2] = t.seconds / 10;
        digits[3] = t.seconds % 10;
    }
}

ISR(TIMER2_OVF_vect){
    counter++;
    if(multiplier == -1 && millis == 0)
        return;
    if (coeff > millis && multiplier == -1){
        millis = 0;
        return_capacity(millis);
        return;
    }
    if(counter != 1 && counter % 200 != 0) return;

    if(counter == 1)
        coeff = 10;
    else if (counter == 1000)
        coeff = 50;
    else if(counter == 1600)
        coeff = 100;
    else if(counter == 2400)
        coeff = 600;
    else if(counter == 3200)
        coeff = 3000;
    else if(counter == 3800)
        coeff = 6000;

    millis = (millis + coeff*multiplier);
    if(millis >= 54000)
        millis = 0;

    return_capacity(millis);
}

void return_number(){
    unsigned char flag = 0;
    unsigned char r = 0;
    cli();
    flag = EEPROM_read(0);
    sei();
    for(unsigned int j = 1; j < 5; j++){
        cli();
        r = EEPROM_read(j);
        sei();
    }
}

```

```

        digits[j-1] = r;
    }

    millis = 0;
    if(flag == 0x0A){
        millis += digits[3];
        millis += digits[2]*10;
        millis += digits[1]*100;
        millis += digits[0]*600;
    }
    else if (flag == 0x0B){
        millis += digits[3] * 10;
        millis += digits[2] * 100;
        millis += digits[1] * 600;
        millis += digits[0] * 6000;
    }
    return_capacity(millis);
}

void write_digit(void)
{
    unsigned char r = 0;
    if(less_10_min) r = 0x0A;
    else r = 0x0B;
    cli();
    EEPROM_write(0, r);
    sei();
    for(int unsigned j = 1; j<5; j++){
        r = digits[j-1];
        cli();
        EEPROM_write(j, r);
        sei();
    }
}

ISR(TIMER1_COMPA_vect){
    if(!millis){
        PORTC|=(1<<3);
        TCCR1B = 0x00;
        return;
    }
    return_capacity(--millis);
}

void button_handler(char port, char new_multipl){
    if(~PINC&(1<<port)){
        multiplier = new_multipl;
        return_capacity(millis);
        TCCR2B = 0b00000011;
        while(~PINC&(1<<port))
            ;
        counter = 0;
        TCCR2B = 0x00;
        write_digit();
    }
}

void init(){
    DDRD = ~0x03;
    DDRB = 0b00111111;
    DDRC = 0b00001000;
    PORTC = 0x07;
}

```

```

sei();
TCCR0B=0b00000001;
TCNT0 = 0;
TIMSK0|=(1<<TOIE0);

//TCCR1B = 0b00001011;
TIMSK1|=(1<<OCIE1A);
TCNT1 = 0;
OCR1A =6250;

//TCCR2B = 0b00000011;
TCNT2 = 0;
TIMSK2|=(1<<TOIE2);

return_number();
}

int main(void)
{
    init();
    while (1)
    {
        button_handler(0, 1);
        button_handler(1, -1);
        if(~PINC&(1<<2)){
            if(TCCR1B == 0x00){
                if(millis>0){
                    PORTC&=~(1<<3);
                    TCCR1B = 0b00001011;
                }
            }
            else
                TCCR1B = 0x00;
            while(~PINC&(1<<2))
                ;
        }
    }
}

```

Приложение 2. Демонстрация работы устройства в симуляторе Proteus

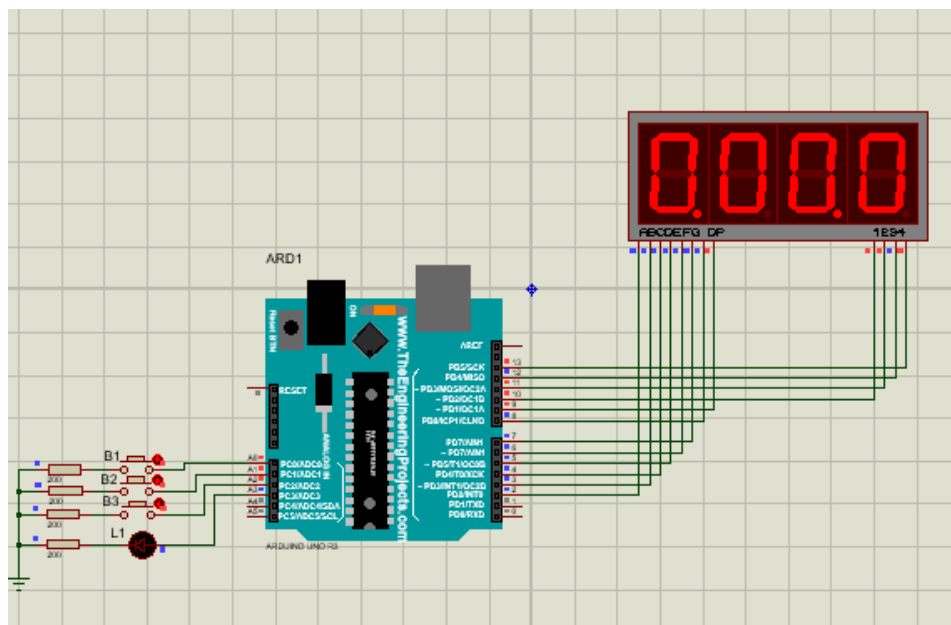


Рис 5. Включение устройства.

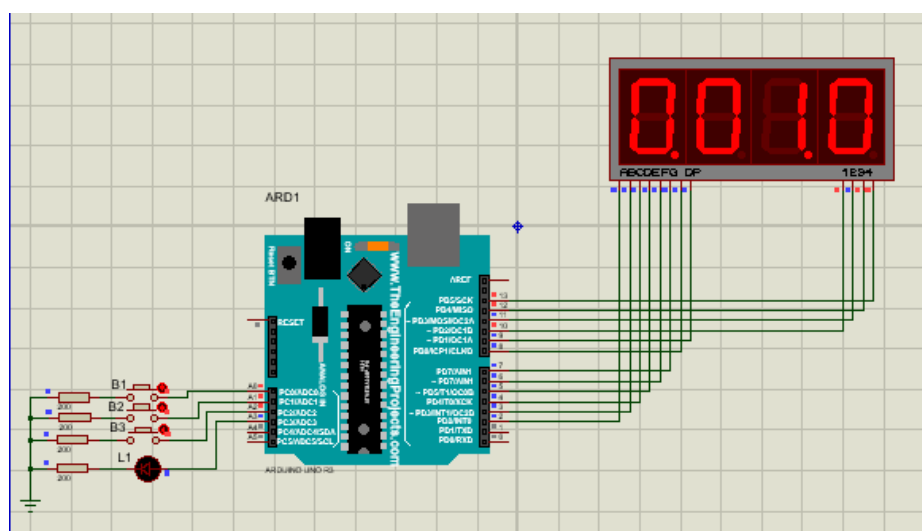


Рис 6. Кратковременное нажатие кнопки В1.

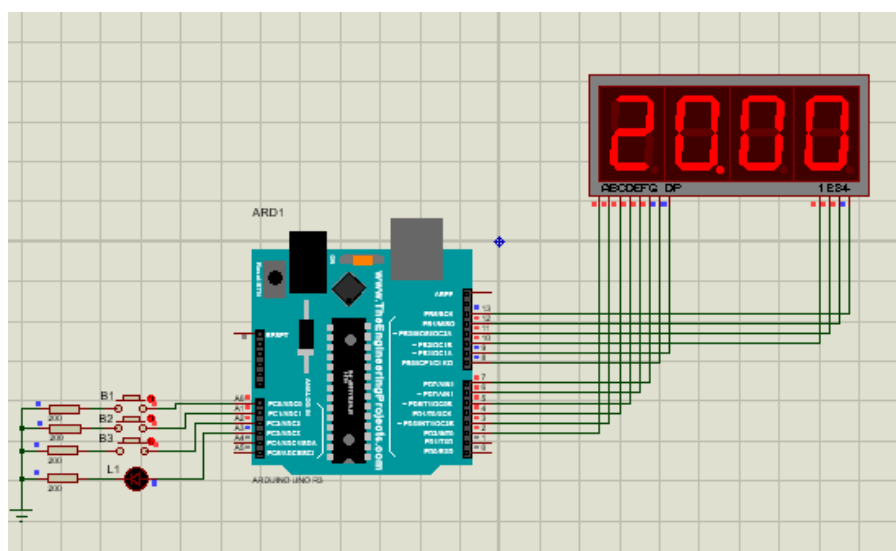


Рис 7. Зажимаем кнопку В1 пока таймер не выдаст 20 минут.

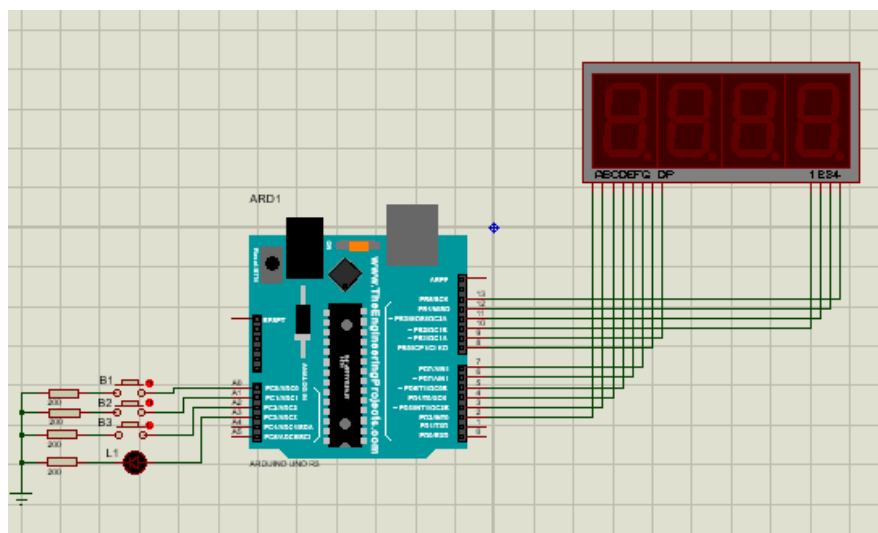


Рис 8. Выключаем устройство.

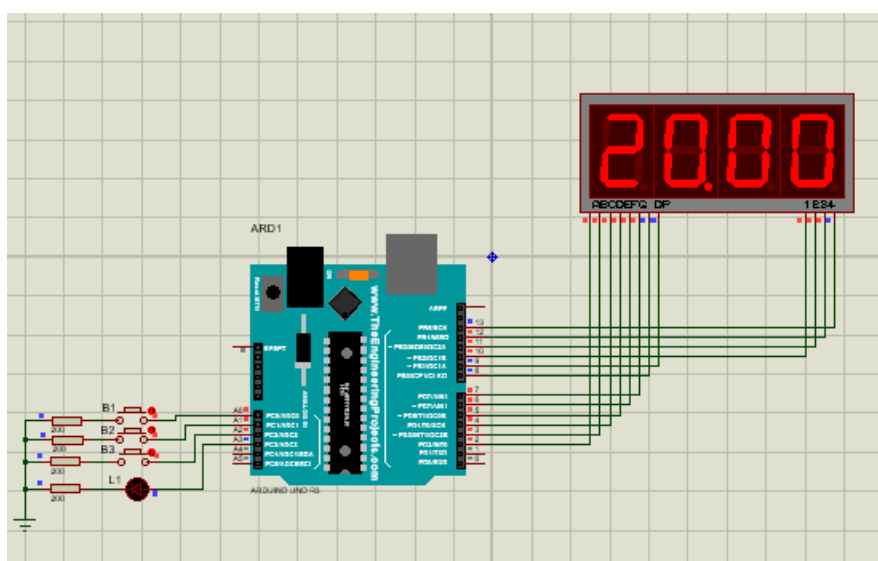


Рис 9. Включаем устройство. Сохранение данных в EEPROM работает.

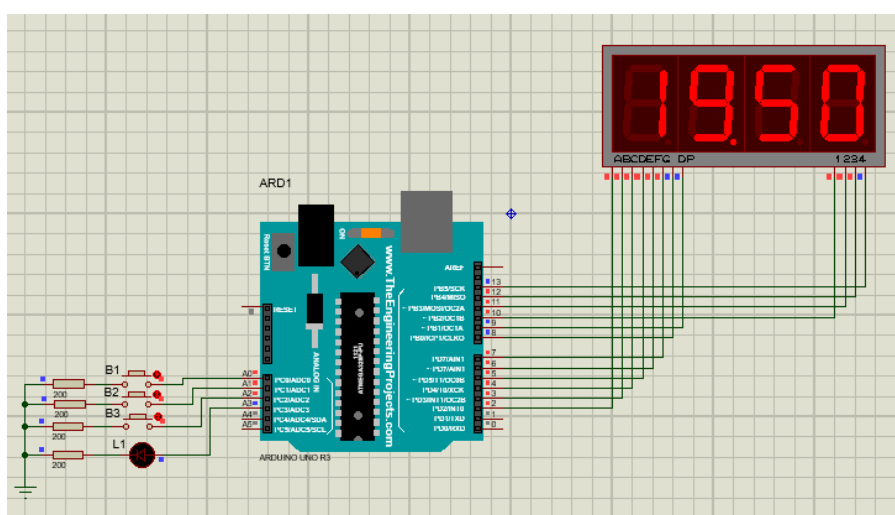


Рис 10. Включим таймер (кнопка B3). Ждем, пока он отсчитает 10 секунд.

Выключим таймер.

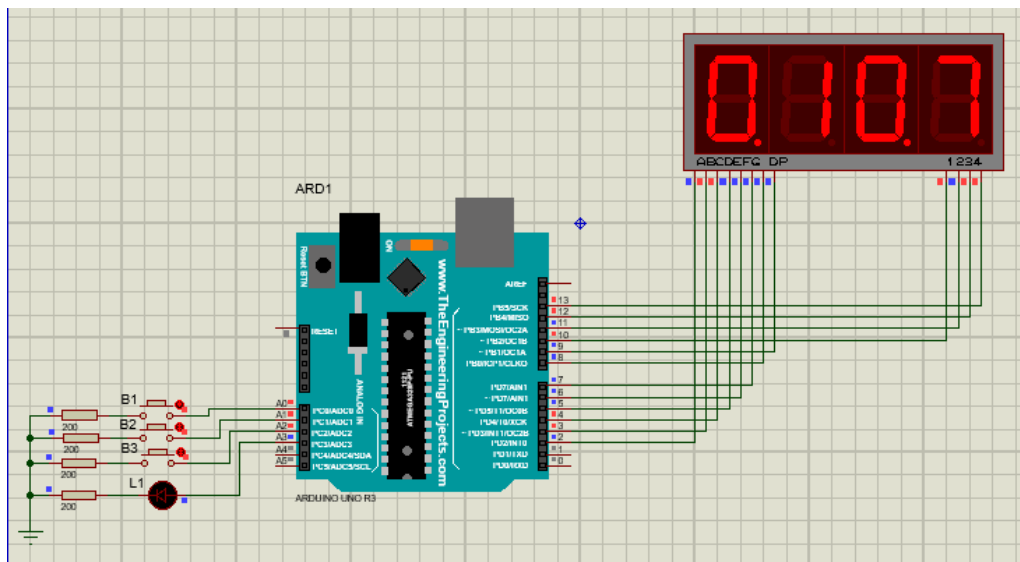


Рис 11. С помощью кнопки B2 уменьшим время таймера до 10 секунд.

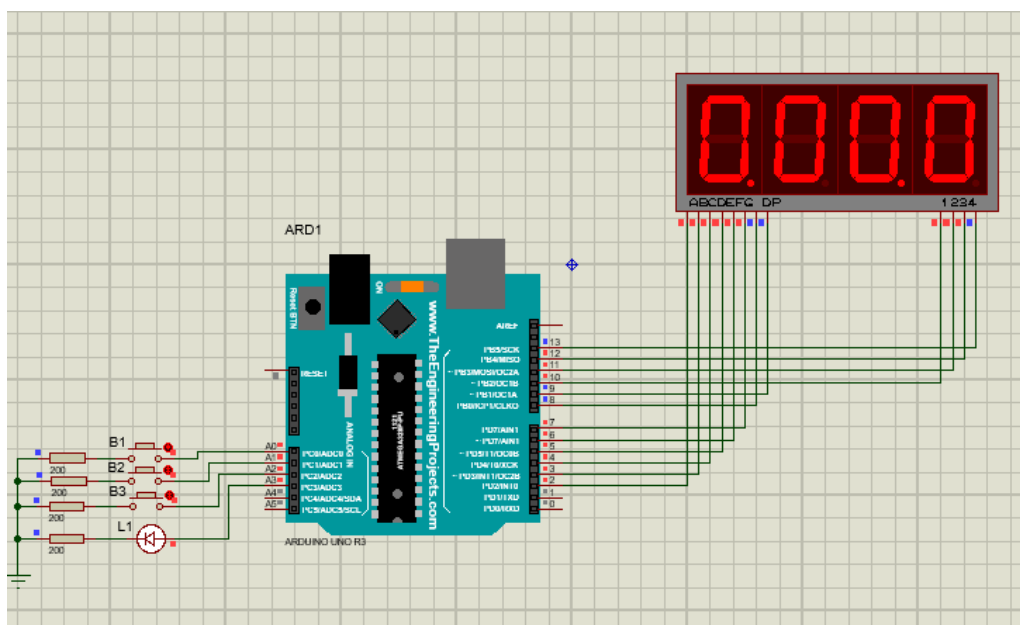


Рис 12. Снова запустим таймер. Ждем, пока он досчитает до 0.

Приложение 3. Демонстрация работы собранного прототипа устройства

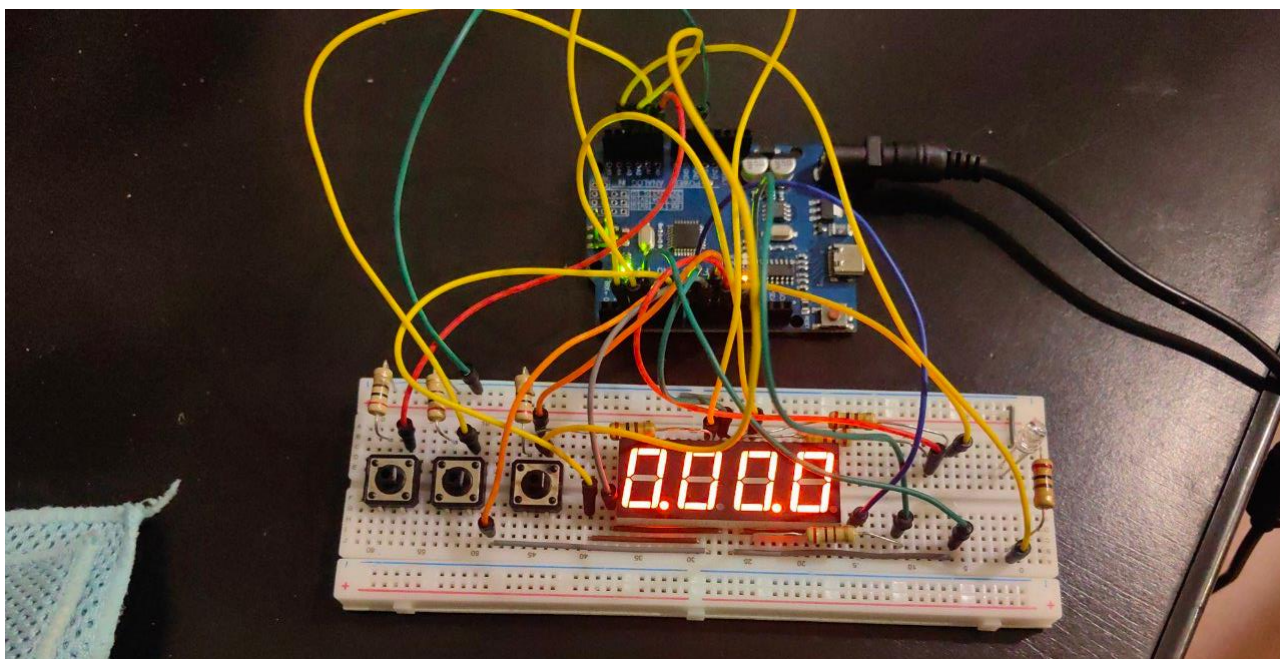


Рис 13. Включение устройства.

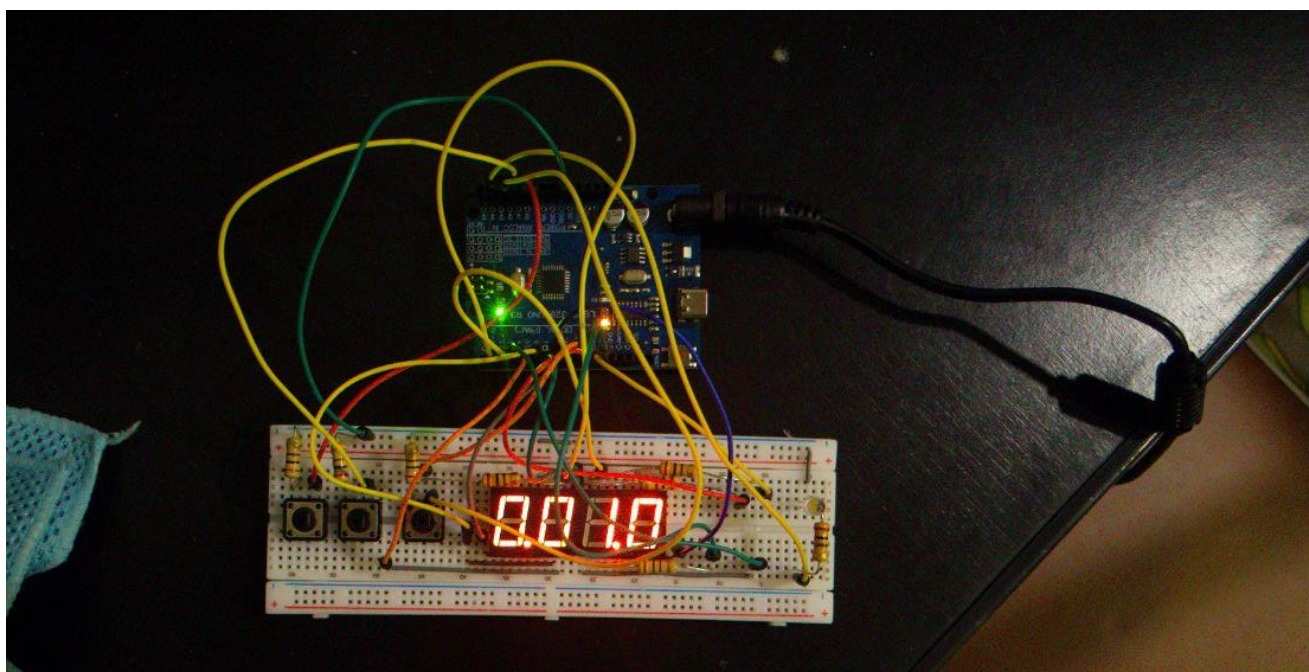


Рис 14. Кратковременное нажатие кнопки В1.

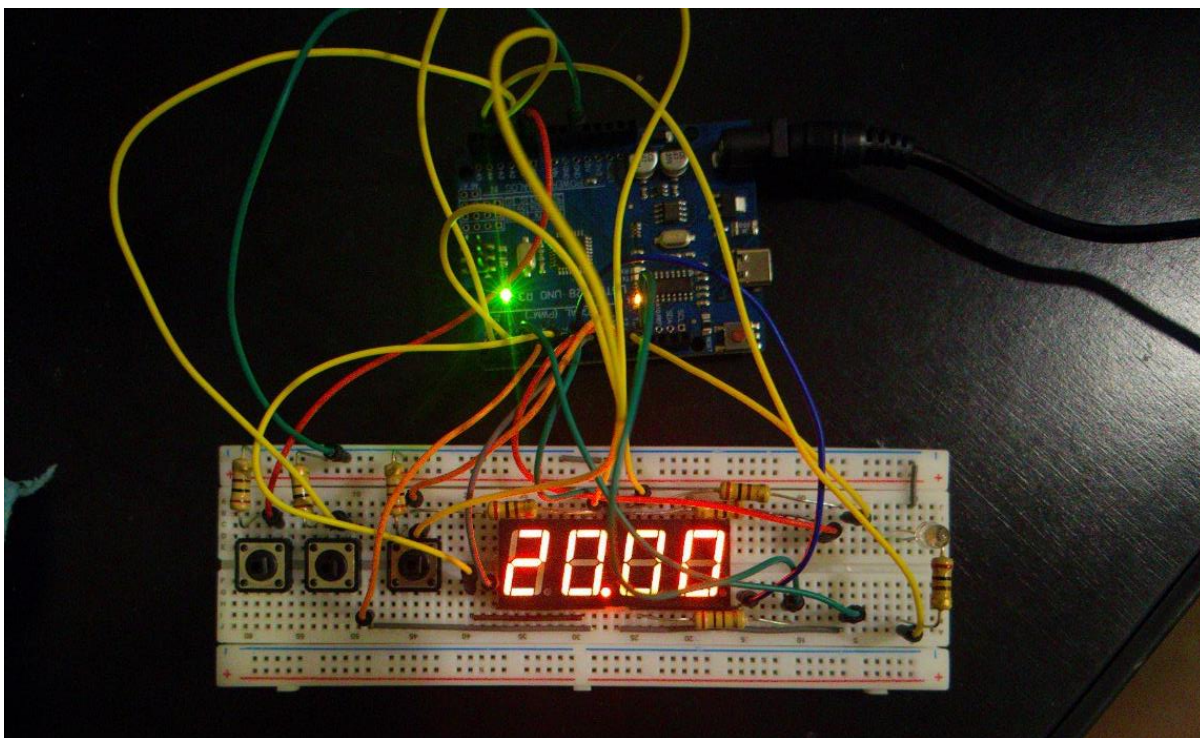


Рис 15. Зажимаем кнопку В1 пока таймер не выдаст 20 минут.

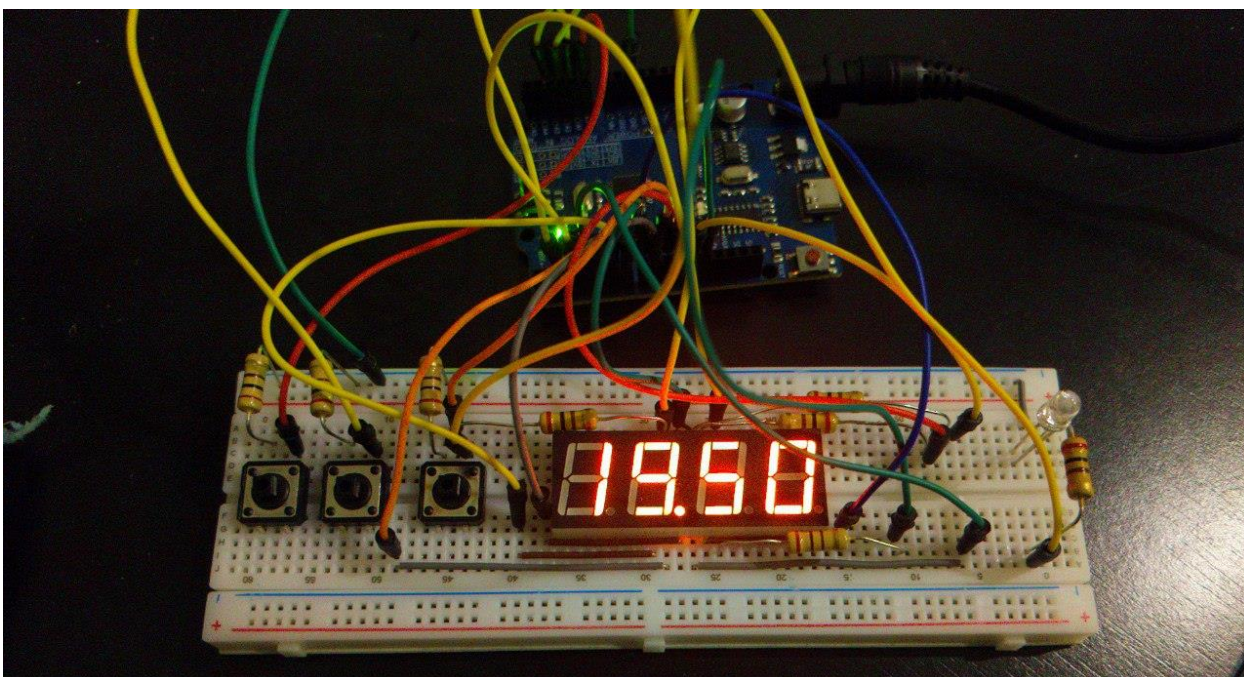


Рис 16. Включим таймер (кнопка В3). Ждем, пока он отсчитает 10 секунд.

Выключим таймер.

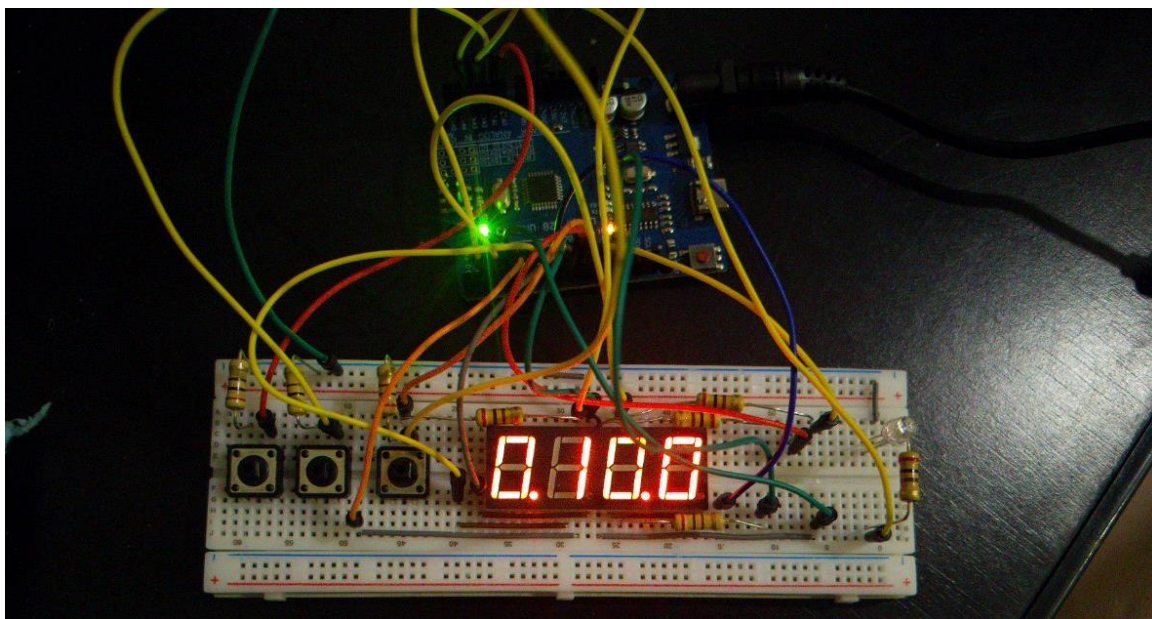


Рис 17. С помощью кнопки В2 уменьшим время таймера до 10 секунд

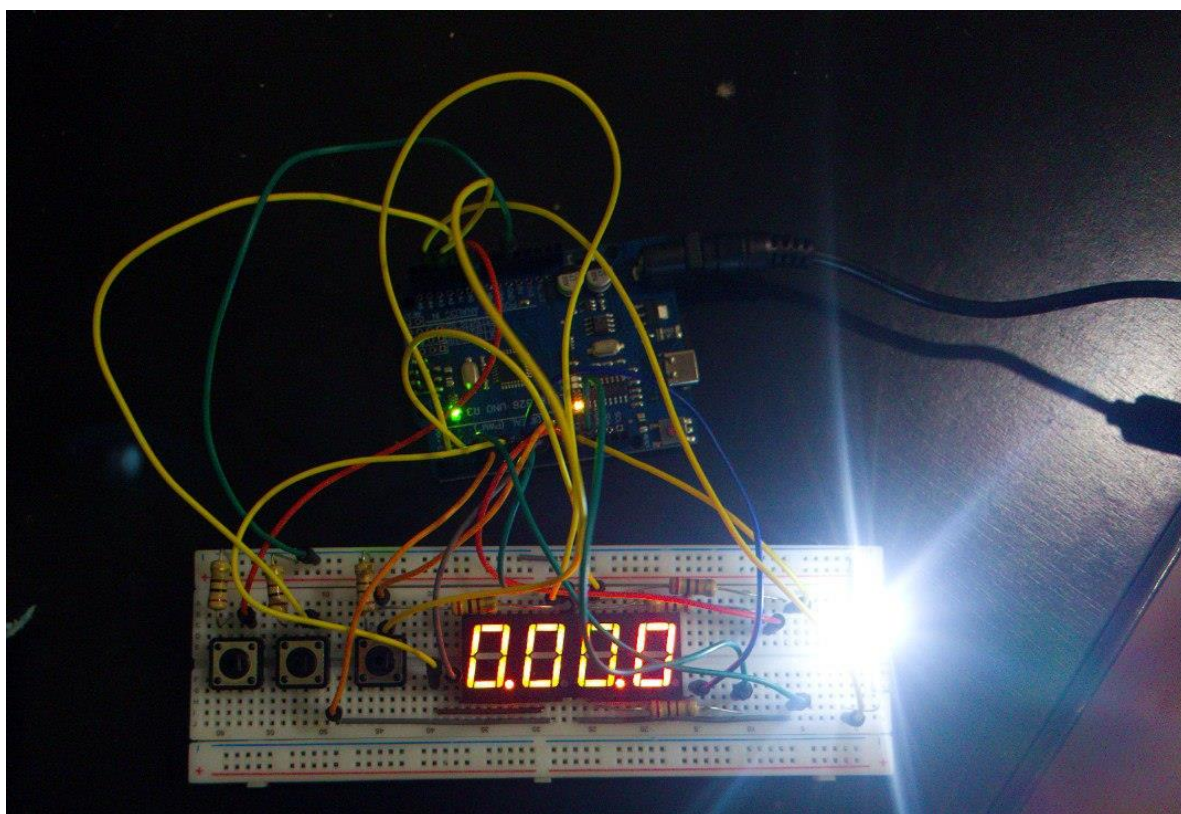


Рис 18. Снова запустим таймер. Ждем, пока он досчитает до 0.

Список использованных источников

1. Arduino Uno Rev3/R3 — Характеристики, описание платы [Электронный ресурс] // <https://micro-pi.ru/>: форум радиоэлектроники. — URL: <https://micro-pi.ru/arduino-uno-rev3-r3-%D0%BE%D0%BF%D0%B8%D1%81%D0%B0%D0%BD%D0%B8%D0%B5-%D0%BF%D0%BB%D0%B0%D1%82%D1%8B/> - Дата публикации: 17.06.2018.
2. Arduino Uno R3 на базе процессора ATmega328 [Электронный ресурс] // <https://arduinoplus.ru/>: форум радиоэлектроники. — URL: <https://arduinoplus.ru/plata-arduino-uno/> - Дата публикации: 19.12.2017
3. 5461AR, Красный семисегментный индикатор / дисплей с общим катодом, 4 цифры [Электронный ресурс]// <https://www.ozon.ru/>: интернет-рынок. - URL: <https://www.ozon.ru/product/5461ar-2sht-krasnyy-semisegmentnyy-indikator-displey-s-obshchim-katodom-4-tsifry-0-56-50-3h19h8mm-1625658655/> (дата обращения 27.04.2025)
4. Бобков, П. Учебный курс AVR. Таймер - счетчик T0. Регистры. Ч1 [Электронный ресурс]// <https://chipenable.ru/>: форум радиоэлектроники. URL <https://chipenable.ru/index.php/programming-avr/171-avr-timer-t0-ch1.html> - Дата публикации: 14.08.2013.
5. ATmega328P – datasheet [текст]// Atmel Corporation URL: <https://www.alldatasheet.com/datasheet-pdf/pdf/241077/ATMEL/ATMEGA328P.html> (дата обращения 27.04.2025)